Developer Guide — Knotted Coil Designer

This guide is for anyone extending the tool. Read it together with:

- `Design.md`, the frozen specification and its changelog;
- `DECISIONS.md`, why things are the way they are;
- `docs/REQUEST_LOG.md`, what was asked for and when.

1. What the tool does, in one paragraph

It designs MRI receive phased arrays made of **flat knotted wire loops** (torus knots T(p,q) drawn as rosettes) on a helmet. The pieces are:

- **Element shape:** knot type, bloom (lobe length), petal sharpness (lobe narrowness), rotation, wire.
- **Electromagnetics:** a quasi-static thin-wire model. It gives mutual and self-inductance (Neumann formula), the coupling coefficient k, and the B1 field (Biot–Savart).
- **Solver:** arranges N elements on the helmet so neighbouring elements are geometrically decoupled (k ≈ 0) while covering the region.
- **Front end:** a single browser page with no build step, plus Node.js tests.

2. Repository map

Path	What it is
`core/knot.js`	Element geometry: `points`, crossings and over/under pattern, lift solver, clearance, bend radius, lobe aspect ratio, bloom floor, adaptive point count
`core/inductance.js`	Neumann mutual inductance (with near-field refinement), self-inductance, Maxwell coaxial closed form, elliptic integrals
`core/surface.js`	Helmet ellipsoid, coverage region, local frames, wrapping a flat element onto the surface
`core/layout.js`	Seeded random numbers; initial even spread of N centres (farthest-point sampling + Lloyd)
`core/optimise.js`	Decoupling solver: stage 1 lookup table, stage 2 exact; pair scan; final report
`core/field.js`	Biot–Savart B1; head surface and slice sampling
`app/index.html`	The whole UI: Element, Pair explorer, Array on helmet, exports. Heavy maths runs in Web Workers
`tests/run.js`	Validation checks V1–V25 (pre-registered; V15–V20 and V25 read from `sim/out/*/cosim.json`) plus labelled post-hoc checks. Writes `validation/`
`tests/compare_worstpair.js`	Pre-registered paired solver comparison (protocol in `validation/`)
`tests/compare_ring_vs_free.js`	Pre-registered comparison C2: ring vs free layouts (v1.6). Run before `tests/run.js`, because V12 reads its arrays
`tests/b1_crosscheck.py`	V14: our B1 against Magpylib 6.0.0 on identical wires. Needs a Python environment with `magpylib==6.0.0`; set `KC_PYTHON` to its Python executable (default `~/.venvs/knotted_coils/Scripts/python.exe` on Windows, `~/.venvs/knotted_coils/bin/python` elsewhere)
`tools/serve.js`	Tiny local web server: `node tools/serve.js` → http://localhost:8765/app/index.html. Also the COMSOL job runner (v1.11)
`core/simcase.js`	v1.11: the app's designs as case files for the full-wave solvers (array, single element, pair sweep)
`sim/comsol_run.py`	v1.11: builds, solves and tunes a case in COMSOL through MPh; writes results the app reads
`tools/build_dist.js`	Builds `dist/`, a flat copy of the app for hosting or sending
`vendor/`	Three.js 0.149.0 (MIT), pinned and stored locally so the app works offline
`docs/`	Request log; copies of the research-integrity rules this project follows

Path	What it is
`sources/`	Archived primary sources for VERIFIED citations

3. How the code is organised

- **Plain JavaScript, no build step, no dependencies** apart from the vendored Three.js.
- **Module pattern:** every `core/*.js` file wraps itself in a factory that
  - attaches to a global `KC` namespace (`KC.knot` , `KC.ind` , `KC.surface` , `KC.layout` , `KC.opt` , `KC.field`);
  - pushes the factory onto `KC_FACTORIES` ;
  - exports `KC` for Node.
- **Same code in Node and the browser.** The app builds its Web Worker by concatenating `KC_FACTORIES[i].toString()` into a Blob, so the worker runs exactly the same code. **Load order matters:** knot, inductance, surface, layout, optimise, field.
- **Units:** mm, nH, μT/A, radians internally (degrees only in the UI).
  - Coordinates: x is left–right, y is back–front, z is up.
- **The element formula** (Design §3.1, v1.4):
  - $\psi = q\varphi + a\cdot\sin(q\varphi)$, with $a = 1 - 1/m$ (sharpness m);
  - $r = R(1 + \beta(1 + \cos\psi)/2)$;
  - $x = r\cdot\cos(p\varphi + \theta)$, $y = r\cdot\sin(p\varphi + \theta)$, $z = h\cdot\sin\psi$.
  - The v1.0 “symmetric” shape ($r = R(1 + \beta\cos q\varphi)$) is still selectable.

v1.6 features, and where they live

- **Face exclusion zones:** `KC.surface.zoneHit` and `zonePoints` . Zones are angular ellipses seen from the helmet centre; defaults are in `surface.DEFAULTS.zones` .
  - The solver keeps wires out with disc samples in the fast stage and real wires in the exact stage. Zones take priority in free-mode moves.
  - `covSamples` drops coverage demands next to zones.
- **Ring mode:** `KC.opt.ringSetup` , `ringLayout` and `ringOptimise` .
  - Each element’s parameters come from `ctx.elemFn(i)` .
  - Left twins use `mirror: true` : a reflected knot, with **points in reverse order** so current circulates the same way on every element.
  - Midline elements get rotations that are multiples of π/q.
- **Multi-start:** `KC.opt.run` with `opts.starts` . The best start is chosen by worst *adjacent* |k| in the exact report.
- **Adjacency:** `KC.opt.adjacency` . Two elements are adjacent when some helmet sample lies on the border between their regions (Voronoi).

- **Capacitor breaks:** `KC.knot.breaks` places them; `KC.knot.capacitor` gives values (γ/2π from CODATA 2022). The app exports them as ports.
- **B1:** `KC.field.vector` uses the exact finite-segment formula, not the midpoint rule; V14 checks it against Magpylib.

v1.7 full-wave capacitor co-simulation (`sim/`)

- **Setup (Windows, outside OneDrive):**
  - openEMS 0.37.0 unpacked to `C:\Users\reza\.solvers\openEMS`; override with `KC_OPENEMS_DIR`.
  - Python 3.14 environment `C:\Users\reza\.venvs\kc_em` with numpy, scipy, h5py, matplotlib, magpylib 6.0.0, and the openEMS/CSXCAD wheels from `openEMS\python`.
  - The scripts set `CSXCAD_INSTALL_PATH` for their own process; no permanent `setx` is used.
  - Large working files go to `C:\Users\reza\.solvers\work` (`KC_SIM_WORK`).
- **Workflow:**
  1. `node tests/export_case.js`, or the app's "Export simulation case", writes `sim/cases/*.json`.
  2. `python sim/run_openems.py sim/cases/<case>.json` runs one FDTD simulation per capacitor break and writes `sim/out/<case>/openems.npz` and `openems_meta.json`.
  3. `python sim/cosim.py <case>` chooses the capacitors and evaluates coupling and B1. It writes `cosim.json` and PNGs.
  4. **Run `loop_air_3T` first at each grid resolution:** it calibrates the thin-wire correction (`sim/out/thinwire_calibration.json`), which every other case uses.
- **Matching topology:** shunt C_m across the feed gap, series L_m to the preamp. The tuning condition is $X = \sqrt{(Z_0 R)}$ (derivation in the `sim/cosim.py` header). L_m·C_m resonance gives preamp decoupling.

v1.8 thickness and v1.9 crossovers

- **Thickness T:** `KC.knot.height(prm, clear, n)` returns `{h, zOff, hMin, thickness, clamped}`. Points take `h` and `zOff` (`z = h sin ψ + zOff`). Arrays use `knot.height` in `cfg.knot`.
- **Crossovers:** `KC.opt.crossovers(wires, normals, clear)` bumps higher-numbered elements outward over lower ones.
  - `O.report` applies it. The report carries `wires` (bumped), `layoutWires` (pre-bump), `kMatrix` (bumped), `kMatrixLayout` and `crossovers`.
  - V9 uses the layout fields; V24 checks the bumps.
- **Full-wave pre-run gate:** `python sim/check_crossings.py sim/cases/<case>.json` must report zero risks before `run_openems.py`. `run_openems.py` also aborts on dropped ports (preflight) and on grid cells < 0.15 mm.

v1.10 COMSOL interface (UNVALIDATED)

- **Shared geometry:** `sim/wirecut.py` (numpy only) cuts wires at capacitor gaps for both solvers, so openEMS and COMSOL see identical ports and wire ends.
- **Export:** `sim/export_comsol.py <case.json>` writes `sim/comsol/<case>/`:
  - `seg_e<element>_<k>.txt`;
  - `ports.csv` (A, B terminals; face direction u);
  - `phantom.json`;
  - `meta.json`.
- **Build:** `sim/comsol_build_mph.py <case>` uses MPh 1.4.0 (MIT; `pip install mph`, which needs JPype and a licensed COMSOL with the RF Module).
  - **Model:** PEC edges for wires, a lumped-port face per gap, nested-ellipsoid phantom, air sphere with a scattering boundary, frequency study.
  - **Solve:** one solve per excited port; S is assembled from lumped-port V and I as in openEMS.
  - **Output:** `sim/out/<case>_comsol/{comsol.npz, comsol_meta.json, model.mph}`.
  - `--dry-run` needs no COMSOL; `--no-solve` builds and saves only.
- **First use on a COMSOL machine:**
  1. `--no-solve`, then open `model.mph` in the GUI.
  2. Check that every wire is one PEC edge set, every port face bridges its gap, and each port's direction runs A → B.
  3. Check the tissue domains and the mesh on the wires.
  4. Fix any API name the script got wrong and **record the fix**.
  5. Solve `loop_air_3T` first: it calibrates the PEC-edge thin-wire correction, just as for openEMS.
  6. Solve `trefoil_air_3T`, run `cosim.py ... --solver comsol`, then `node tests/run.js` for **V25**.

v1.11 COMSOL link in the app (PROPOSED, pending Reza's approval; EXPLORATORY)

The app can send a design to COMSOL, have it solved in the background, and show the results next to its own quasi-static numbers (Design §6d).

- **What runs where:**
  1. The page builds the case files in a worker (`core/simcase.js`).
  2. It posts them to `tools/serve.js`.
  3. The server starts `sim/comsol_run.py --job sim/comsol_jobs/<job>`, one job at a time, detached, so a job survives a server restart.
  4. The script writes `status.json`, `<case>_results.json`, `<case>_stage1.npz` and `<case>.mph` (solution cleared) into the job folder. The page polls the status and reads the results.
- **Set-up (once per machine):** a licensed COMSOL with the RF Module, and a Python environment with MPh (`pip install mph numpy`; MPh also installs JPype).
  - **Windows:**

```
py -m venv %USERPROFILE%\comsolai
%USERPROFILE%\comsolai\Scripts\pip install mph numpy
```

- **Linux / macOS:**

```
python3 -m venv ~/comsolai
~/comsolai/bin/pip install mph numpy
```

- **How the server finds Python:**
  1. `KC_COMSOL_PYTHON`, if set;
  2. otherwise the first that exists of `~/comsolai`, `~/.venvs/kc_em` and `~/.venvs/knotted_coils` (`Scripts\python.exe` on Windows, `bin/python` elsewhere);
  3. otherwise `python` (Windows) or `python3` on the PATH. The app shows which Python it found and whether it can import MPh.
- MPh finds COMSOL itself (default install folders, or the Windows registry).

- **Use:**
  1. Run `node tools/serve.js` and open http://localhost:8765/app/index.html. The COMSOL link does not work when the page is opened as a file.
  2. Set Field strength to 3 T or 7 T (the head phantom has tissue values only there).
  3. In the Array tab's "COMSOL" card, choose:
     - the solved array: 1–2 coils are tuned and matched in COMSOL; 3 or more coils get one port each (feed gap only, no capacitors) and give B1 maps and coupling, without tuning or matching;
     - or the current element alone.
  4. In the Pair explorer, "Verify in COMSOL" runs the pair at several spacings around the zero.
  5. Every case is checked first (pre-flight): coils stretched > 15% by wrapping, wires inside the head, gaps within 1.5 mm of another coil, or wires that would touch are refused with a message saying which coil. Results of earlier runs open from "previous runs…"; one COMSOL run at a time (the buttons grey out), and Cancel stops it.
- **Without the app:** `python sim/comsol_run.py case.json [--mode auto|b1|ports|match] [--build-only] [--no-insulation]` (auto: `match` for 1–2 coils, `b1` for arrays). "Download case + STL" in the card writes the case file. The STL is for viewing only; COMSOL is built from the exact wire paths in the case file.
- **Timing measured on the development copy** (24 cores, 16 used, normal mesh):
  - single trefoil in the head: 2–4 minutes (about 15–20 s per port);
  - two coils: about 13 minutes per spacing (about 65 s per port, 350k mesh elements).
  - 8-coil array, B1 only: about 16 minutes (7 min geometry and mesh, then about 1 min per coil; 431k mesh elements).
- **Things learned while building it (2026-10-23; details in DECISIONS):**
  - COMSOL's lumped ports put a 1 V incident wave on the driven port, so $S_{kp} = V_k - \delta_{kp}$.

- The study must exist before the physics-controlled mesh, because mesh sizes depend on the frequency.
- A failed mesh leaves a stale partial mesh: clear it before retrying, and accept a mesh only if every domain holds elements.
- Insulation beads must extend 0.6 mm or more beyond the wire, or the mesher fails.
- Draw the tube through the actual wire points (spline interpolation functions). A global Fourier series (Will's method) misses sharp crossover bumps by millimetres and makes geometry building slow.
- Select domains by the geometry object that made them (`selresult`), not with ball selections: those need a display mesh of every face and fail on large models.
- The tube is many separately fitted surface patches. On an 8-coil array with all 32 gaps, a few patch joins did not meet exactly and the mesh failed; with feed gaps only it meshes. Capacitors in array models would need one swept tube per coil.

4. Common extensions — where to start

You want to…	Do this
Add a knot family (Lissajous, Fourier)	Add a generator next to `knot.points` that returns the same `[[x,y,z],…]` format, with a z-lift that encodes over/under. Everything downstream (crossings, lift, inductance, solver, B1) works on point lists. Add a V-check: crossing count and pattern equal the family's known diagram
Use a real helmet former	Replace the ellipsoid in `core/surface.js` : `normal` , `snap` , `alongNormal` , `inCoverage` , `samples` , `coveredArea` , `point` . An STL importer would need nearest-point projection and normals
Mix element types in one array	`optimise.js` assumes one shared element shape (one lookup table, one self-inductance). Store per-element shape parameters; build one table per distinct pair of shapes
Improve the solver	It is a transparent accept-if-better local search and is **strongly seed-sensitive** (DECISIONS.md). Cheapest gain: multi-seed restarts, keeping the best. Any change → pre-register a paired comparison like `validation/COMPARE_worstpair_PROTOCOL.md`
Better field maps	Export wire paths (CSV, OBJ/STL) to a full-wave solver: openEMS (free), CST, HFSS, Sim4Life. Don't add E-field maps to this quasi-static model (DECISIONS.md explains why)
Add an export	`app/index.html` , "exports" section. `arrRes` holds the solved array (`wires` , `centres` , `theta` , `kMatrix` , `pairs` , `config`)

5. Rules this project follows (please keep them)

These come from the research-integrity framework in `docs/research_integrity/` :

- **Freeze, then change by changelog.** `Design.md` is frozen. Any change to geometry, the EM model or validation criteria gets a version bump and a dated changelog entry saying who asked, why, and what it affects.

- **Pre-register checks.** Write the pass criterion into `Design.md` §7 (or a protocol file) and **commit before running**. Report failures as failures: the git history keeps every failing run, e.g. runs 1, 4 and 5. Never edit a criterion after seeing results. Fixes to the test harness are logged as such.
- **Tag every parameter's origin** (LIT / CONV / JUDG / TUNED / DERIVED, with PLACEHOLDER where relevant), inline in the code *and* in `PARAMETERS.md`.
- **Cite only VERIFIED sources.** Read the primary text and archive it in `sources/`. Label anything else UNVERIFIED.
- **Provenance header** on every source file (AI-assisted authorship, accountable person).
- **Label results exploratory** unless validated to the tier the claim needs. Every coupling and B1 output says so.
- **Decision log:** dated entries in `DECISIONS.md`, including mistakes and how they were found.

6. Run, test, build

```
node tools/serve.js
```

Then open http://localhost:8765/app/index.html. Double-clicking `app/index.html` may also work, but this hasn't been tested.

```
node tests/run.js
```

Validation suite, about 5–15 minutes. Writes `validation/validation_report.{html,json}` and `validation_status.js`, which drives the app's header banner.

```
node tests/compare_worstpair.js
```

Re-runs the pre-registered worst-pair comparison, about 3–5 minutes.

```
node tools/build_dist.js
```

Builds `dist/`, a flat copy of the app for hosting.

7. Known limitations and open items (as of 2026-10-21)

- **The design tool is quasi-static.** The v1.7 full-wave path (`sim/`) adds capacitors, tissue, wave effects and preamplifier decoupling, but only for small cases, and is unvalidated against measurement.
- **Placeholders:** wire spec and helmet dimensions (PARAMETERS.md).
- **V3:** cannot be evaluated until Roemer et al. 1990 is read (paywalled).
- **V6:** fails below the wire-dependent bloom floor. It is a known physical limit, shown in the UI.
- **Solver:** seed-sensitive. No 32-element test array met $|k| < 0.01$ for all nearest pairs.
- **Crossovers in dense arrays (v1.9):** bumps work for pairs, but in heavily overlapping knot arrays they stack up to the 10 mm cap and some crossings cannot be separated (V24 FAIL on arrays). Such arrays are flagged "not buildable as drawn". Thicker knots (T) need far fewer bumps. Open design problem.
- **Very sharp petals** (high m, 3–5 windings) need crossing lifts of several mm.

- **Untested:** opening the app from `file://` (double-click). The local server is tested.
- **v1.11 COMSOL link:** tested on Linux with COMSOL 6.4 and MPh 1.3.1.
  - The Windows paths (Python discovery, hidden job windows, `taskkill` cancel, UTF-8 files) were written for Windows but not run there.
  - Wires are perfect conductors (no copper loss).
  - The coupling k at f0 with shorted gaps includes wave effects, so it is not a clean check of the Neumann engine.